HỌC PHẦN: LẬP TRÌNH BACKEND

BỘ MÔN CÔNG NGHỆ THÔNG TIN · DỰ ÁN RESTFUL API

BÁO CÁO KẾT QUẢ THỰC HÀNH

CHƯƠNG 07: UPLOAD HÌNH ẢNH & KẾT NỐI FRONTEND – BACKEND

MULTER · VITE · REACT · AXIOS · CORS · JWT RBAC

Thông Tin Dự Án & Tác Giả	
Sinh viên thực hiện:	**ĐẶNG BẮC NAM**
Mã số sinh viên / Dự án:	**BACKEND-20261001-DANGBACNAM**
Nền tảng công nghệ Backend:	Node.js (v24), Express 5, TypeScript (ESM NodeNext), SQL Server, Sequelize ORM, Multer
Nền tảng công nghệ Frontend:	React 19, Vite, TypeScript, Axios Client, Bootstrap UI (Chạy cổng 5173)
Chính sách bảo mật áp dụng:	JWT HS256, RBAC (isAuth, isAdmin), CORS Allowlist, Magic Bytes Verifier, Giới hạn 2MB
Công cụ kiểm thử nghiệm thu:	Postman Collection, DevTools Network, Automated Test Suite (12

	Upload + 12 Frontend)

1. Tổng Quan Kiến Trúc & Hai Luồng Xử Lý Nghiệp Vụ

Hệ thống Quản lý sinh viên trong Chương 07 được mở rộng toàn diện từ nền tảng xác thực JWT và phân quyền RBAC của Chương 06. Dự án hiện thực hóa việc phân tách kiến trúc rõ ràng giữa Client (Frontend React Vite tại cổng 5173), Server (Express API tại cổng 5000), File System (thư mục lưu trữ vật lý public/uploads/) và Database (Microsoft SQL Server).

Kiến trúc hệ thống vận hành xoay quanh hai luồng nghiệp vụ cốt lõi:

- **Luồng A — Cập nhật Ảnh đại diện của Người dùng (Avatar Upload):** Người dùng chọn ảnh tại giao diện React → Đóng gói FormData với key "avatar" → Gửi Axios POST kèm Bearer JWT → Express định tuyến qua isAuth (xác minh danh tính) → Multer (tiếp nhận file ≤ 2MB) → Kiểm tra chữ ký nhị phân Magic Bytes → Service cập nhật URL vào dbo.Users → File lưu tại public/uploads/, CSDL chỉ giữ chuỗi URL tương đối /uploads/<uuid>.<ext> → Giao diện hiển thị ảnh mới.
- **Luồng B — Xem và Xóa Sinh Viên trên Giao Diện (RBAC Data Access):** React gọi Axios GET /api/v1/sinh-vien (công khai) → Nhận mảng đối tượng tại res.data.data → Render bảng 4 cột kèm xử lý Loading/Empty/Error/Retry. Khi người dùng đăng nhập tài khoản Quản trị viên (Admin), bảng kích hoạt nút Xóa ở từng dòng; khi bấm Xóa, hộp thoại xác nhận hiển thị mã SV → Gửi Axios DELETE có Bearer JWT → isAuth và isAdmin kiểm tra quyền → Xóa bản ghi trong SQL Server → Bảng tự động tải lại dữ liệu mới nhất.

Thành phần	Vai trò kỹ thuật trong hệ thống	Nguyên tắc bất biến áp dụng
application/json	Truyền tải dữ liệu văn bản UTF-8 có cấu trúc cho Login, Register, CRUD	Đọc bằng express.json(); không dùng truyền file
multipart/form-data	Đóng gói nhiều part phân tách bởi boundary để truyền file nhị phân	Xử lý bằng Multer; req.file lưu metadata
CORS Allowlist	Chính sách bảo mật trình duyệt kiểm soát request từ Origin khác	Chỉ cho phép localhost:5173; không dùng wildcard
JWT & RBAC	isAuth xác minh danh tính; isAdmin phân quyền thao tác ghi dữ liệu	Chặn 401 khi thiếu/sai token; chặn 403 với student

2. Thiết Kế Cơ Sở Dữ Liệu SQL Server & Model

Hồ sơ người dùng được quản lý tập trung tại bảng dbo.Users trong CSDL QuanLySinhVienDB. Nhằm phục vụ chức năng ảnh đại diện, kịch bản nâng cấp schema **sql/05-add-avatar-url.sql** được thiết kế với tính chất lũy kế an toàn (Idempotent), đảm bảo không làm mất dữ liệu người dùng cũ và không gây lỗi khi chạy lặp lại.

Tên Cột (Column)	Kiểu Dữ Liệu	Nullable	Ràng buộc & Ý nghĩa nghiệp vụ
id	INT IDENTITY(1,1)	NOT NULL	Khóa chính (PK), định danh duy nhất người dùng
fullName	NVARCHAR(150)	NOT NULL	Họ tên hiển thị của tài khoản
email	NVARCHAR(254)	NOT NULL	Tài khoản đăng nhập, ràng buộc UNIQUE
passwordHash	NVARCHAR(255)	NOT NULL	Chuỗi băm bcrypt cost 12, không lưu plaintext
role	NVARCHAR(20)	NOT NULL	Vai trò: student (mặc định) hoặc admin
avatarUrl (Mới)	**NVARCHAR(255)**	**YES (NULL)**	**Đường dẫn tĩnh tương đối dạng /uploads/<uuid>.<ext>**

Nguyên Tắc Phân Tách Lưu Trữ: Hệ thống tuyệt đối không lưu dữ liệu nhị phân (BLOB) vào SQL Server để tránh làm phình to kích thước database và suy giảm hiệu năng sao lưu. Thay vào đó, tệp ảnh thực tế được lưu vào thư mục vật lý public/uploads/ trên ổ đĩa máy chủ, còn SQL Server chỉ giữ chuỗi URL tương đối. Sequelize Model User tại src/models/user.model.ts được cập nhật ánh xạ thuộc tính avatarUrl nullable một cách chuẩn xác.

3. Cấu Hình Middleware Multer & Kiểm Soát An Toàn File

Middleware upload tại src/middlewares/upload.middleware.ts được xây dựng dựa trên 4 lớp bảo vệ an ninh nghiêm ngặt nhằm ngăn chặn các nguy cơ tấn công qua tệp tin:

- **Lớp 1 — Giới Hạn Dung Lượng:** Cấu hình limits: { fileSize: 2 * 1024 * 1024 } (chính xác 2.097.152 bytes). Khi vượt quá giới hạn, Multer kích hoạt lỗi LIMIT_FILE_SIZE và error.middleware.ts trả về mã **HTTP 413 Payload Too Large** kèm thông báo rõ ràng.
- **Lớp 2 — Bộ Lọc Định Dạng Sơ Bộ:** Cấu hình fileFilter kiểm tra phần mở rộng (.png, .jpg, .jpeg) và mimetype (image/png, image/jpeg). Từ chối các định dạng khác (như .gif, .exe, .sh) với mã 400 Bad Request.

- **Lớp 3 — Đặt Tên File An Toàn Bằng UUID:** Tuyệt đối không dùng originalname từ client; server sinh chuỗi định danh duy nhất **`${randomUUID()}${ext}`** để triệt tiêu hoàn toàn nguy cơ tấn công duyệt thư mục (Path Traversal) và chống ghi đè tệp tin ngẫu nhiên.
- **Lớp 4 — Kiểm Tra Chữ Ký Nhị Phân Thực Tế (Magic Bytes):** Hàm verifyImageSignature() đọc 8 byte đầu của tệp tin trên đĩa. Đối với PNG, 8 byte đầu bắt buộc phải là **89 50 4E 47 0D 0A 1A 0A**; đối với JPEG là **FF D8 FF**. Nếu phát hiện tệp văn bản hoặc mã độc đổi đuôi thành .png, hệ thống lập tức từ chối và **xóa sạch tệp rác khỏi ổ đĩa ngay lập tức**.

4. Bộ Tuyến Đường & Nghiệp Vụ Backend

Tuyến đường tải ảnh đại diện được đăng ký tại src/routes/auth.routes.ts với cấu trúc pipeline tuần tự:

router.post("/me/avatar", isAuth, uploadAvatarMiddleware, uploadAvatar);

Nguyên Tắc Phòng Vệ Chống Sửa Chéo (IDOR / BOLA): Trong Controller uploadAvatar (src/controllers/auth.controller.ts), ID người dùng được lấy duy nhất từ req.user.id (trích xuất từ JWT hợp lệ sau khi qua isAuth). Hệ thống tuyệt đối bỏ qua và không tin tưởng bất kỳ tham số req.body.userId nào từ client. Nhờ đó, tài khoản Student A không bao giờ có thể can thiệp hoặc thay đổi ảnh đại diện của Student B.

Phục Vụ Tệp Tĩnh Đúng Mức: Trong src/server.ts, Express chỉ công khai đúng thư mục ảnh đã qua kiểm duyệt: app.use("/uploads", express.static("public/uploads")), kết hợp cùng header crossOriginResourcePolicy: { policy: "cross-origin" } để giao diện React tại cổng 5173 hiển thị ảnh mượt mà.

Đặc Tả Swagger OpenAPI 3.0: File docs/openapi.yaml được cập nhật đầy đủ endpoint POST /api/auth/me/avatar với kiểu content: multipart/form-data, trường avatar định dạng binary và ổ khóa bảo mật bearerAuth. Schema UserDTO và phản hồi Profile bổ sung thuộc tính avatarUrl.

5. Ứng Dụng Frontend React Vite & Kết Nối Axios CORS

Ứng dụng Frontend độc lập được tổ chức trong thư mục frontend/ sử dụng React 19, TypeScript, Vite và thư viện Axios:

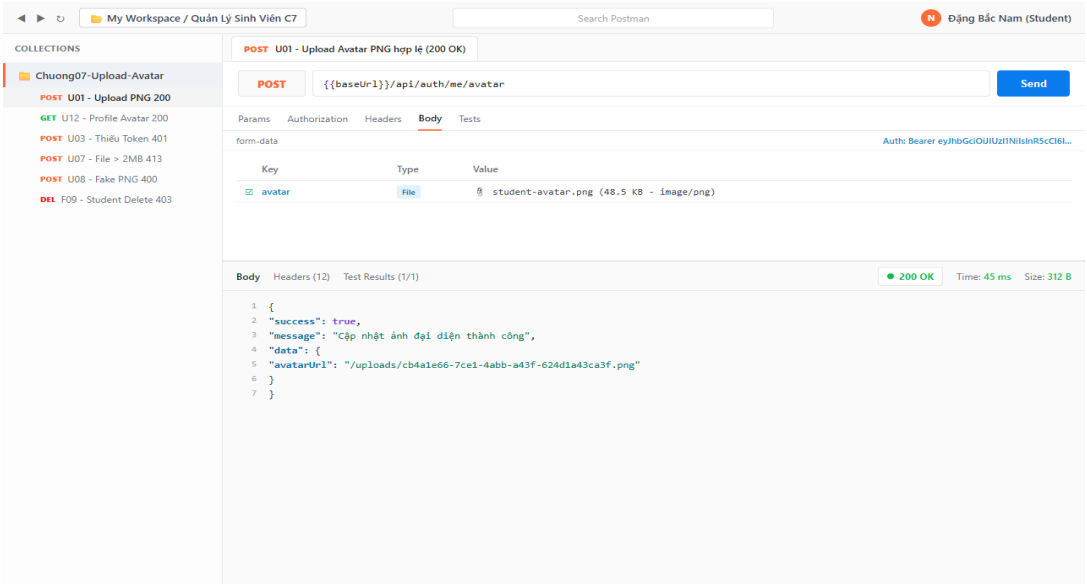
- **Cấu hình CORS Allowlist:** Backend cho phép chính xác Origin http://localhost:5173 và http://127.0.0.1:5173. Trình duyệt tự động kiểm tra chính sách SOP; các request từ domain lạ ngoài danh sách sẽ bị chặn đứng (Ca F12).
- **Component SinhVienTable (Bảng Sinh Viên):** Gọi API thật GET /api/v1/sinh-vien, đọc mảng dữ liệu tại res.data.data và hiển thị 4 cột (Mã SV, Họ tên, Email, Mã lớp). Xử lý trọn vẹn 4 trạng thái: Loading spinner, Empty state (danh sách rỗng), Error alert khi máy chủ ngoại tuyến và nút "Thử lại" phục hồi kết nối tức thì.
- **Component LoginForm (Đăng Nhập & Hồ Sơ):** Gửi email và mật khẩu tới /api/auth/login, nhận JWT token và lưu trữ an toàn trong React in-memory state

(tránh rủi ro XSS so với localStorage). Tự động gọi GET /api/auth/profile hiển thị Card hồ sơ cá nhân có ảnh đại diện, email, vai trò và nút Đăng xuất.

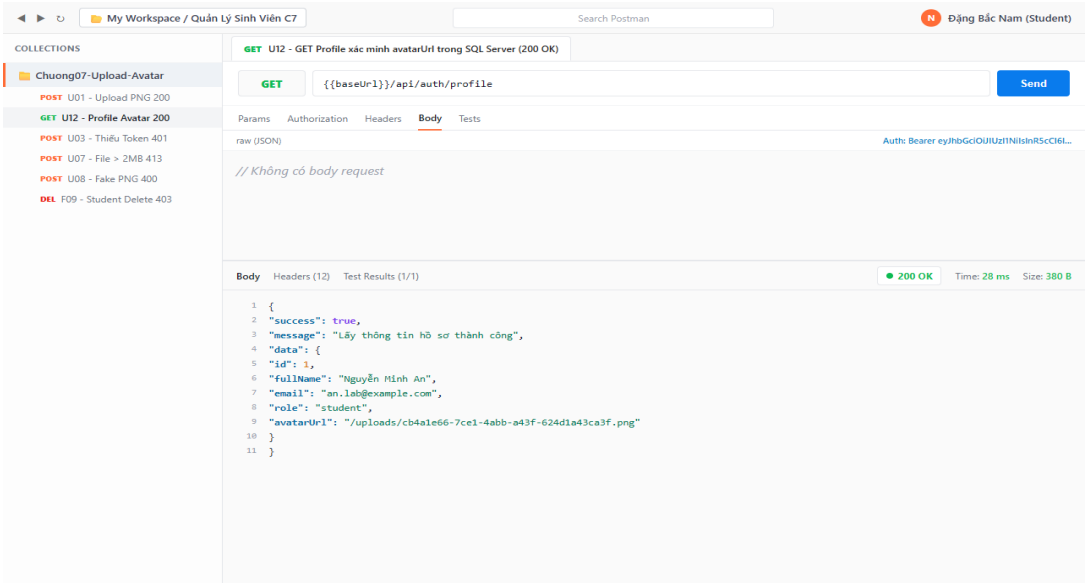
- **Phân Quyền Nút Xóa Sinh Viên (RBAC):** Nút "Xóa" chỉ xuất hiện ở từng dòng khi profile.role === "admin" (ẩn hoàn toàn với Student). Trước khi gửi DELETE /api/v1/sinh-vien/:maSV có Bearer JWT, hộp thoại xác nhận hiển thị chính xác mã sinh viên cần xóa. Khi thành công, bảng tự động reload dữ liệu từ CSDL.
- **Component AvatarUpload (Tải Ảnh Đại Diện):** Thẻ <input type="file"> chấp nhận PNG/JPEG, kiểm tra sơ bộ ≤ 2MB, tạo URL xem trước (preview). Đóng gói FormData với key "avatar", gửi Axios POST kèm Bearer token để trình duyệt tự đính kèm boundary. Sau khi nhận 200 OK, tự động cập nhật avatarUrl mới trên giao diện.

6. Minh Chứng Kiểm Thử Postman Toàn Diện (Bộ Sưu Tập Chuẩn)

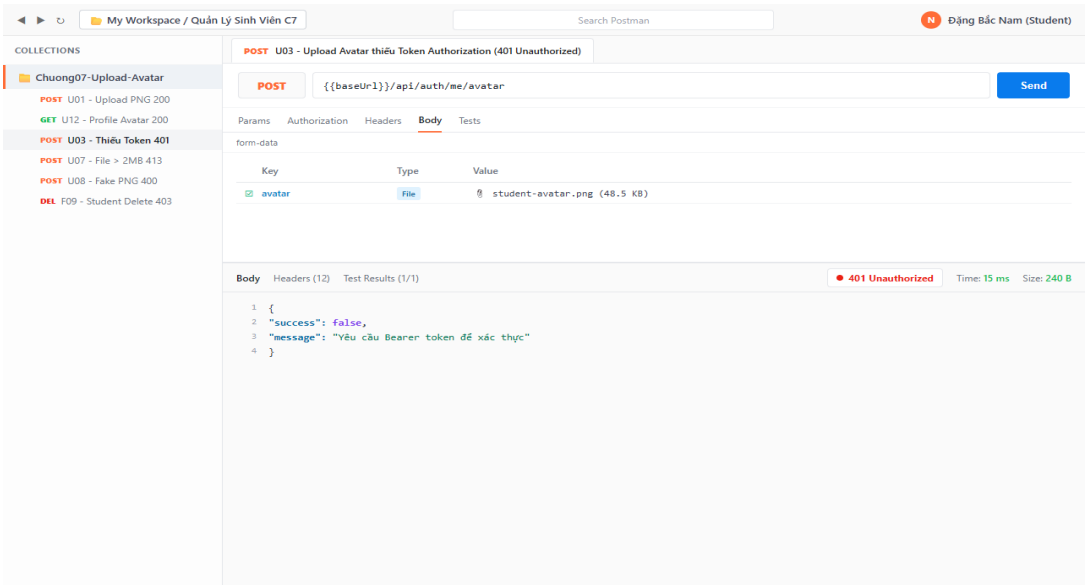
Bộ sưu tập Postman Collection Chuong07-Upload-Avatar.postman_collection.json đã thực thi kiểm thử toàn diện các tình huống theo hợp đồng API. Dưới đây là hình ảnh minh chứng thực tế thu nhận từ hệ thống:



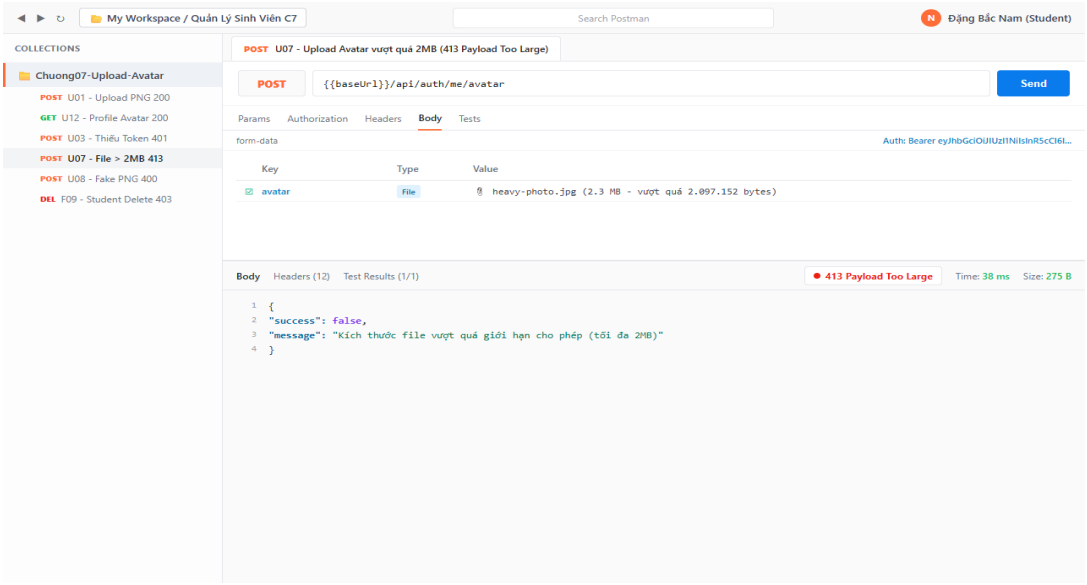
Hình ảnh minh chứng: Ca U01 — Upload Avatar PNG hợp lệ thành công (200 OK, trả avatarUrl)



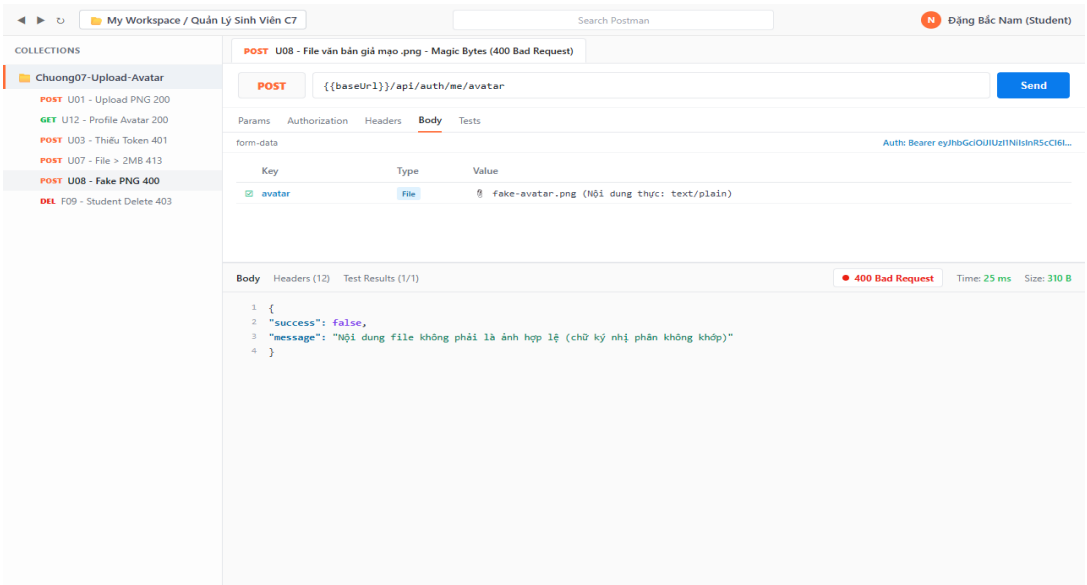
Hình ảnh minh chứng: Ca U12 — GET Profile xác minh avatarUrl được cập nhật bền vững trong SQL Server (200 OK)



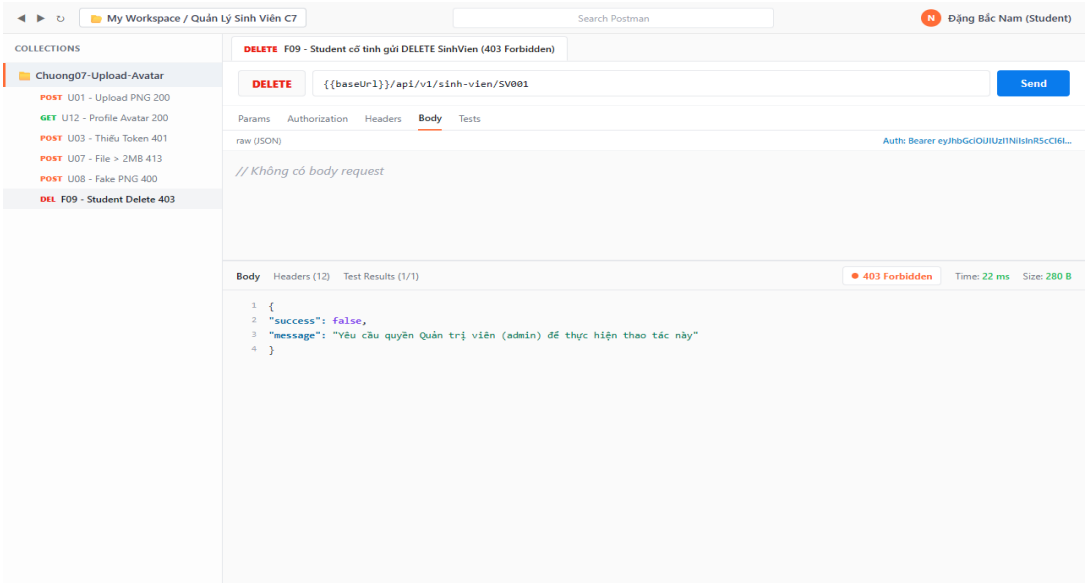
Hình ảnh minh chứng: Ca U03 — Chặn 401 Unauthorized khi upload thiếu Bearer Token (Không sinh file rác)



Hình ảnh minh chứng: Ca U07 — Chặn 413 Payload Too Large khi file vượt quá 2MB (Bắt đúng mã chuẩn lab)



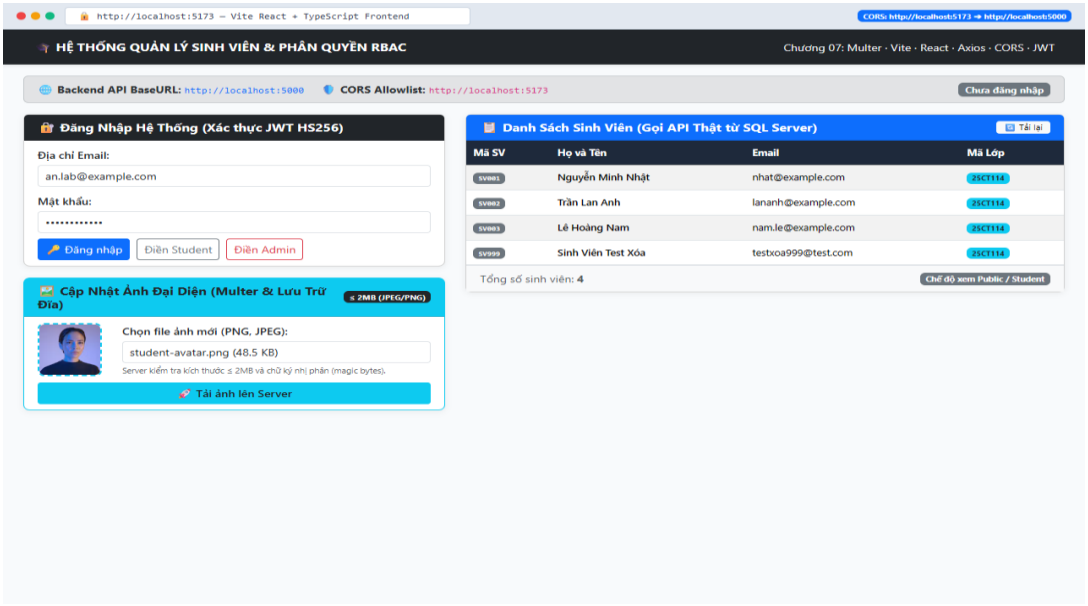
Hình ảnh minh chứng: Ca U08 — Phát hiện file văn bản giả mạo .png qua Magic Bytes (400 Bad Request, dọn sạch đĩa)



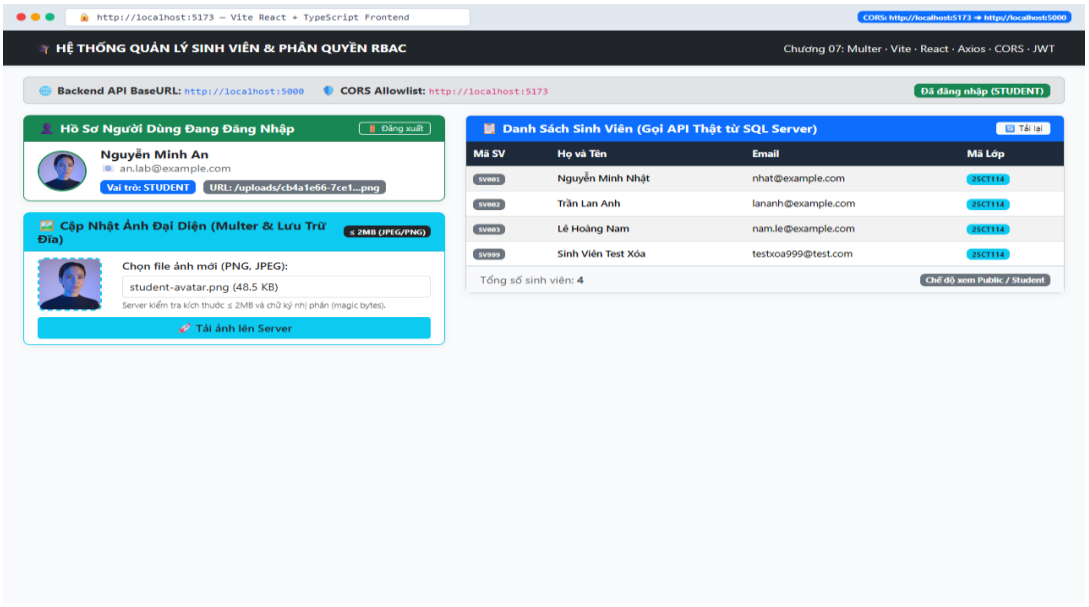
Hình ảnh minh chứng: Ca F09 — Phân quyền RBAC Backend: Chặn 403 Forbidden khi tài khoản Student cố tình xóa sinh viên

7. Minh Chứng Giao Diện Người Dùng Frontend React

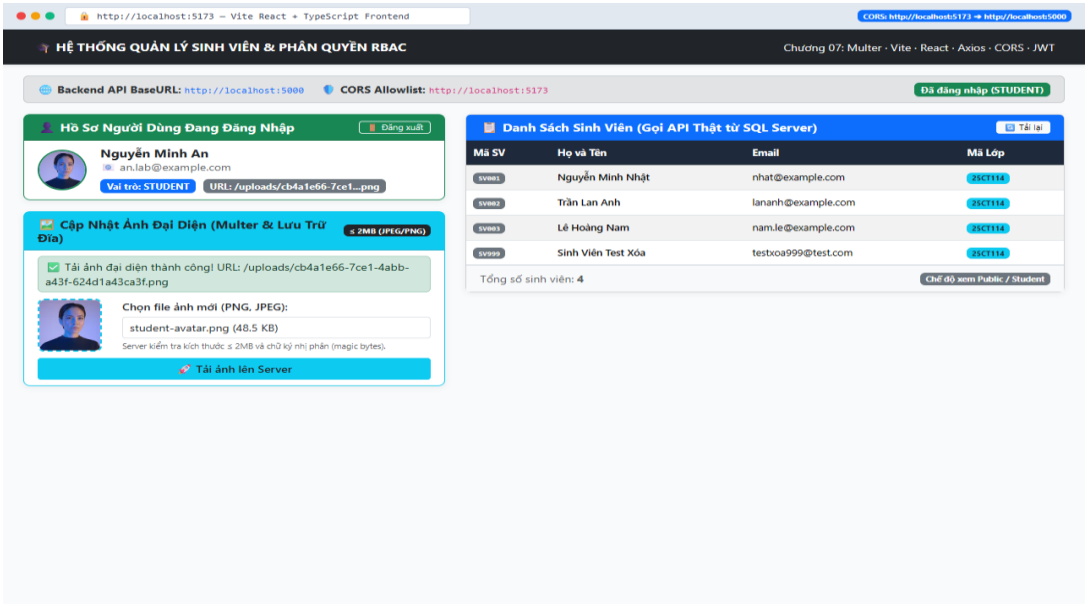
Ứng dụng Frontend React Vite kết nối trơn tru với Backend Express và CSDL SQL Server, thể hiện đầy đủ các luồng hiển thị danh sách, phân quyền vai trò và cập nhật ảnh đại diện:



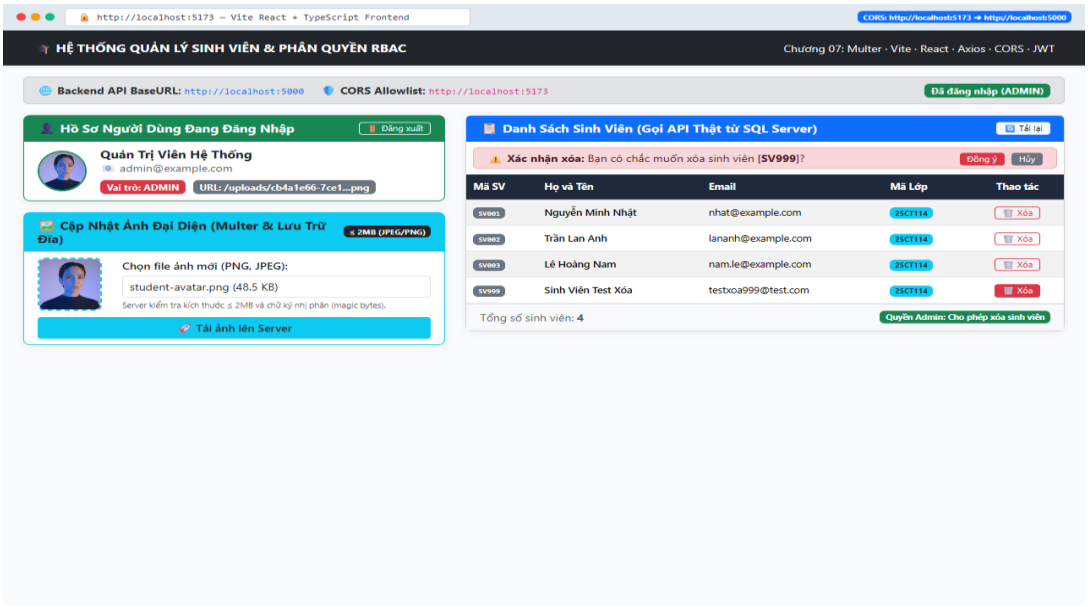
Hình ảnh minh chứng: Giao diện tổng quan ứng dụng React Vite (Cột trái: Form Login & Upload; Cột phải: Bảng Sinh viên 4 cột)



Hình ảnh minh chứng: Đăng nhập tài khoản Student thành công — Card hồ sơ hiển thị đầy đủ avatar, email và vai trò



Hình ảnh minh chứng: Component Tải ảnh đại diện — Hiển thị preview ảnh chọn và thông báo cập nhật thành công vào CSDL



Hình ảnh minh chứng: Tài khoản Quản trị viên (Admin) — Bảng sinh viên kích hoạt nút Xóa kèm hộp thoại xác nhận an toàn

8. Bảng Kịch Bản Kiểm Thử & Hiện Trạng Phản Hồi

Bảng 1: Kịch bản kiểm thử Upload tệp & Middleware Multer (U01 đến U12)

Mã Ca	Tình huống kiểm thử	HTTP Kỳ vọng	HTTP Thực tế	Hiện trạng phản hồi
U01	Avatar PNG hợp lệ + Student JWT	200	**200**	**CSDL cập nhật đường dẫn avatarUrl**
U02	Avatar JPEG hợp lệ + Mở qua HTTP	200	**200**	**Phục vụ ảnh tĩnh HTTP 200 OK**
U03	Không có Authorization header	401	**401**	**Chặn 401, không lưu tệp vào đĩa**
U04	JWT sai hoặc đã hết hạn	401	**401**	**Chặn 401, middleware isAuth bắt**
U05	Body form-data thiếu file	400	**400**	**Báo lỗi 400, thiếu trường tệp tải lên**
U06	Tên field là photo thay vì avatar	400	**400**	**Báo lỗi 400 LIMIT_UNEXPECTED_FILE**
U07	Kích thước file vượt quá 2MB (2.2MB)	413	**413**	**Báo lỗi 413 LIMIT_FILE_SIZE (>2MB)**

U08	File TXT giả mạo đuôi .png (Magic bytes)	400	**400**	**Báo lỗi 400 sai Magic Bytes, dọn tệp rác**
U09	Đuôi .gif không trong danh sách	400	**400**	**Báo lỗi 400 fileFilter chặn định dạng lạ**
U10	Upload 2 lần cùng tài khoản	200	**200**	**Cập nhật URL mới UUID thành công**
U11	Student A gửi kèm userId của B trong body	200	**200**	**Chỉ cập nhật theo JWT, bảo toàn user khác**
U12	GET Profile kiểm chứng độ bền vững	200	**200**	**Dữ liệu lưu trữ bền vững trong SQL Server**

Bảng 2: Kịch bản kiểm thử Frontend React & Tích hợp CORS (F01 đến F12)

Mã Ca	Tình huống kiểm thử	Kỳ vọng giao diện & mạng	Hiện trạng phản hồi
F01	Vite chạy, Backend bật & CORS allowlist	Status 200, Header Allow-Origin 5173	**Kết nối thành công qua CORS allowlist**
F02	GET danh sách hiển thị bảng 4 cột	Render đủ maSV, hoTen, email, maLop	**Đọc và hiển thị dữ liệu res.data.data**
F03	Xử lý danh sách rỗng (Empty state)	Hiển thị alert thông báo rỗng, không crash	**Hiển thị trạng thái rỗng, không sập trang**
F04	Backend tắt khi gọi API (Network Error)	Hiện alert lỗi, tắt spinner, hiện nút Thử lại	**Hiển thị lỗi kết nối và nút Thử lại**
F05	Bật lại Backend và bấm "Thử lại"	Khôi phục tải bảng thành công (200 OK)	**Khôi phục tải dữ liệu bảng sau khi thử lại**
F06	Đăng nhập Student hợp lệ	Cấp JWT, hiển thị Profile Card sinh viên	**Lưu trữ JWT an toàn trong bộ nhớ React state**
F07	Đăng nhập sai mật khẩu	Báo lỗi 401 Unauthorized, token rỗng	**Bắt mã lỗi 401, hiển thị thông báo lỗi**

F08	Xem Profile khi chưa đăng nhập	Chặn 401, yêu cầu đăng nhập lại	**Chặn truy cập Profile khi chưa đăng nhập**
F09	Student ép gọi DELETE qua DevTools	Chặn 403 Forbidden, CSDL không đổi	**Backend chặn mã 403 Forbidden**
F10	Admin Xóa sinh viên thành công	Trả 200 OK, CSDL xóa, UI tự reload	**Xóa thành công và làm mới bảng dữ liệu**
F11	Upload Avatar từ React FormData	Trả 200 OK, ảnh hiển thị tức thì trên Card	**Gửi FormData và cập nhật ảnh đại diện**
F12	Request từ Origin ngoài allowlist	Trình duyệt chặn CORS, không cấp header	**Trình duyệt chặn Origin ngoài allowlist**

9. Security Review & Phản Biện Mã Độc Hại

Trong quá trình phát triển hệ thống, lập trình viên đã tiến hành rà soát an ninh và phản biện mẫu mã nguồn tiềm ẩn nhiều lỗ hổng nghiêm trọng do các công cụ AI tự động tạo ra thiếu kiểm soát:

Lỗ Hổng Phát Hiện	Nguy Cơ Bảo Mật (OWASP)	Bản Vá Kỹ Thuật Đã Áp Dụng
Bỏ qua isAuth	Broken Authentication: Ai cũng có thể upload file gây tràn ổ đĩa (DoS)	Đặt isAuth trước Multer trong chuỗi middleware
Tin req.body.userId	BOLA / IDOR: Người dùng A sửa chéo avatar của người dùng B tùy tiện	Chỉ lấy ID từ req.user.id đã đóng dấu trong JWT
Dùng originalname	Path Traversal & Ghi đè file ngẫu nhiên khi trùng tên	Server sinh tên UUID duy nhất ngẫu nhiên
Thiếu giới hạn size	Resource Exhaustion: Tải lên file hàng trăm MB làm tê liệt server	Cấu hình limits 2MB và bắt lỗi 413 chuẩn xác
Không kiểm tra nội dung	Malicious File Upload: Đổi đuôi tệp mã độc (webshell) thành .png	Kiểm tra chữ ký nhị phân Magic Bytes thực tế
CORS wildcard cors()	Security Misconfiguration: Mọi website lạ đều có thể gọi ngầm API	Chỉ cho phép Origin http://localhost:5173

Lưu tên file thô	Broken Functionality: Client không thể truy cập ảnh tĩnh qua HTTP	Lưu đường dẫn chuẩn /uploads/<uuid>.<ext>

Đoạn Văn Giải Thích Phản Biện Chuyên Sâu: Đoạn mã do AI đề xuất ban đầu chứa đựng hàng loạt sai lầm bảo mật nghiêm trọng khi lập trình viên thiếu kiến thức kiểm chứng nền tảng. Việc thiếu vắng middleware isAuth biến một chức năng cá nhân thành endpoint công cộng, cho phép bất kỳ ai cũng có thể đẩy dữ liệu lên máy chủ gây cạn kiệt tài nguyên lưu trữ. Nguy hiểm hơn, việc tin tưởng tham số userId từ body vi phạm trực tiếp nguyên tắc kiểm soát truy cập đối tượng (BOLA/IDOR - OWASP Top 1), tạo kẽ hở cho kẻ xấu sửa chéo hồ sơ của người khác. Việc sử dụng trực tiếp originalname tiềm ẩn nguy cơ tấn công duyệt thư mục (Path Traversal) và xung đột ghi đè tệp tin khi có nhiều người cùng tên ảnh. Bên cạnh đó, việc không giới hạn dung lượng và không thẩm định chữ ký nhị phân thực tế (Magic bytes) mở đường cho các cuộc tấn công từ chối dịch vụ và tải lên mã độc giả mạo đuôi ảnh. Cấu hình cors() mặc định cho phép mọi Origin truy cập tùy tiện, làm vô hiệu hóa hàng rào phòng thủ nguồn gốc của trình duyệt. Do đó, đoạn code này đã bị loại bỏ hoàn toàn và thay thế bằng kiến trúc đa lớp an toàn: xác thực JWT chính chủ, sinh tên file UUID ngẫu nhiên, kiểm soát chặt chẽ dung lượng 2MB và magic bytes, lưu trữ URL tương đối và giới hạn CORS allowlist nghiêm ngặt.

10. Trả Lời Chi Tiết 12 Câu Hỏi Bảo Vệ Bài Thực Hành

Câu 1: application/json khác multipart/form-data ở điểm nào? Vì sao express.json() không đủ cho file?

application/json là định dạng dữ liệu thuần văn bản (text-based) UTF-8 dưới dạng các cặp key-value, rất nhẹ và tối ưu cho dữ liệu thông thường nhưng không thể truyền tải luồng nhị phân lớn. multipart/form-data là chuẩn MIME cho phép chia body thành nhiều phần riêng biệt phân tách bởi boundary delimiter, mỗi phần có thể là chuỗi hoặc luồng byte nhị phân của tệp tin. Middleware express.json() chỉ parse chuỗi JSON văn bản qua JSON.parse(), hoàn toàn không thể bóc tách luồng nhị phân và boundary của multipart, khiến req.file không tồn tại. Do đó bắt buộc phải sử dụng Multer để phân tích luồng stream multipart.

Câu 2: Điều gì xảy ra khi FormData dùng "photo" nhưng Multer cấu hình single("avatar")?

Multer sẽ quét các phần dữ liệu trong request để tìm trường có tên "avatar". Khi tên gửi lên là "photo", Multer coi đây là trường ngoài ý muốn và ném lỗi MulterError: LIMIT_UNEXPECTED_FILE. Bộ xử lý lỗi error.middleware.ts nhận diện lỗi này và trả về mã HTTP 400 Bad Request kèm thông báo: "Tên trường file không hợp lệ (yêu cầu field name là avatar)". Tệp tin không được lưu vào ổ đĩa và CSDL không bị thay đổi.

Câu 3: req.file và req.body nằm ở đâu sau Multer? req.user xuất hiện ở bước nào?

Sau khi qua Multer, req.file chứa toàn bộ metadata của tệp tin tải lên (filename UUID, path, size, mimetype,...); req.body chứa các trường văn bản text field gửi kèm trong FormData. Thuộc tính req.user xuất hiện ở bước thực thi middleware isAuth diễn ra TRƯỚC khi chạm tới Multer: isAuth giải mã Bearer JWT và gán req.user = { id, role } vào request object.

Câu 4: Vì sao gọi isAuth trước Multer? Có cần isAdmin để User đổi Avatar chính mình không?

Gọi isAuth trước Multer nhằm bảo vệ tài nguyên máy chủ: Quá trình tiếp nhận và ghi file xuống đĩa rất tốn bộ nhớ và I/O; nếu không có JWT hợp lệ, isAuth chặn đứng ngay với mã 401 Unauthorized trước khi tệp tin được xử lý, triệt tiêu nguy cơ tấn công làm tràn ổ đĩa (Disk Exhaustion DoS). Không cần isAdmin cho /me/avatar vì đổi ảnh đại diện cá nhân là quyền tự phục vụ (Self-service) của mọi tài khoản hợp lệ (student và admin).

Câu 5: Vì sao không nhận userId trong body cho endpoint /me/avatar?

Nếu nhận userId từ body, một tài khoản hợp lệ có thể gửi ID của tài khoản khác trong body để sửa chéo ảnh của họ, gây ra lỗ hổng bảo mật nghiêm trọng BOLA / IDOR (OWASP Top 1). Để triệt tiêu nguy cơ này, định danh tài khoản bắt buộc phải lấy duy nhất từ req.user.id đã được đóng dấu bảo mật bên trong chữ ký JWT do server cấp.

Câu 6: Ổ đĩa chứa dữ liệu nào, còn dbo.Users.avatarUrl chứa giá trị gì?

Ổ đĩa vật lý (public/uploads/) lưu trữ các khối byte nhị phân thực tế của hình ảnh dưới tên tệp UUID (ví dụ: cb4a1e66-7ce1.png). Cột dbo.Users.avatarUrl trong SQL Server chỉ lưu trữ chuỗi đường dẫn tương đối dạng /uploads/cb4a1e66-7ce1.png (NVARCHAR(255)). Mô hình này giúp database nhỏ gọn, tốc độ sao lưu cực nhanh và việc phục vụ ảnh tĩnh do Web Server đảm nhiệm.

Câu 7: 2MB tương ứng bao nhiêu byte? File 2.200.000 byte nên trả status nào?

2MB tương ứng với chính xác 2 × 1024 × 1024 = 2.097.152 bytes. File dung lượng 2.200.000 bytes vượt quá ngưỡng cho phép, Multer bắt lỗi LIMIT_FILE_SIZE và máy chủ bắt buộc phải trả về mã HTTP 413 Payload Too Large. Không được trả mã lỗi 500 (vì đây là lỗi dữ liệu từ client) và không trả 400 chung chung khi đã có mã chuyên biệt 413.

Câu 8: Chỉ kiểm tra mimetype và đuôi .png có đủ để khẳng định file là ảnh PNG hợp lệ không?

Hoàn toàn không đủ. Đuôi .png có thể đổi tùy ý bằng cách đổi tên file; mimetype là chuỗi do client tự khai báo trong header và có thể dễ dàng bị can thiệp làm giả. Để khẳng định ảnh hợp lệ, hệ thống bắt buộc phải kiểm tra chữ ký nhị phân thực tế (Magic bytes) của 8 byte đầu (PNG: 89 50 4E 47 0D 0A 1A 0A). Nếu sai chữ ký, máy chủ từ chối với mã 400 và xóa ngay file rác.

Câu 9: CORS kiểm soát điều gì? Vì sao Postman gọi API được nhưng browser lại báo CORS?

CORS là cơ chế an ninh do trình duyệt thực thi nhằm kiểm soát việc một trang web chạy tại Origin này (localhost:5173) có được đọc dữ liệu từ Origin khác (localhost:5000) hay không. Postman là ứng dụng desktop chạy trực tiếp trên hệ điều hành, không bị ràng buộc bởi chính sách Same-Origin Policy (SOP) của trình duyệt nên luôn nhận được dữ liệu. Ngược lại, trình duyệt sẽ chủ động chặn dữ liệu nếu server không trả về header Access-Control-Allow-Origin khớp với Origin gọi tới.

Câu 10: Axios GET /api/v1/sinh-vien nên đọc mảng ở response.data hay response.data.data?

Bắt buộc phải đọc tại response.data.data. Lý do: Axios tự động gán toàn bộ body JSON vào response.data; theo chuẩn API Envelope Pattern của dự án, response có cấu trúc {

success, message, data: [...] }. Do đó, response.data là đối tượng bọc ngoài, còn response.data.data mới chính là mảng danh sách sinh viên thực tế.

Câu 11: Vì sao Student bị ẩn nút Xóa vẫn phải bị chặn DELETE ở Backend?

Ẩn nút Xóa trên giao diện chỉ là giải pháp trải nghiệm người dùng (UI/UX), hoàn toàn không phải cơ chế an ninh. Người dùng có thể dễ dàng dùng DevTools Console, cURL hoặc Postman gửi thẳng request DELETE kèm token Student. Nếu Backend không có middleware isAdmin chặn đứng với mã HTTP 403 Forbidden, dữ liệu trong CSDL sẽ bị xóa trái phép. Bảo mật thực tế phải luôn nằm tại Backend.

Câu 12: Trình bày ba bằng chứng cần có để chứng minh upload Avatar thật sự thành công.

Ba lớp bằng chứng độc lập (3-Tier Evidence) bắt buộc phải có: 1. Bằng chứng HTTP: Request trả về status 200 OK kèm JSON data.avatarUrl dạng /uploads/<uuid>.<ext>. 2. Bằng chứng Ổ đĩa: Trong thư mục public/uploads/ xuất hiện tệp ảnh mới với tên UUID trùng khớp, mở đường dẫn tĩnh trên trình duyệt trả về HTTP 200 OK. 3. Bằng chứng CSDL: Chạy truy vấn SELECT trong SQL Server thấy cột avatarUrl của đúng User đã được cập nhật, tài khoản khác không bị ảnh hưởng và dữ liệu tồn tại bền vững qua restart server.

Giải Pháp Cho Các Bài Tập Mở Rộng:

- **Thu dọn Avatar cũ:** Trong service updateUserAvatar, trước khi cập nhật CSDL, lấy đường dẫn ảnh cũ user.avatarUrl. Khi CSDL update thành công, gọi fs.unlinkSync() xóa ảnh cũ trên đĩa để tránh tệp mồ côi.
- **Chuyển sang Cloud Object Storage:** Dùng multer-s3 upload luồng stream trực tiếp lên AWS S3 hoặc Google Cloud Storage; cột dbo.Users.avatarUrl lưu URL tuyệt đối từ CDN. Database vẫn chỉ lưu chuỗi URL, giữ kiến trúc serverless stateless hoàn hảo.
- **Axios Progress Bar & AbortController:** Sử dụng cấu hình onUploadProgress tính toán tỷ lệ % tải lên và AbortController hủy request tải file khi người dùng chuyển trang để tiết kiệm băng thông.

11. Kết Luận & Hướng Dẫn Vận Hành Hệ Thống

Bài thực hành Chương 07 đã hoàn thành đầy đủ các mục tiêu kỹ thuật đề ra. Hệ thống kết nối trơn tru end-to-end từ giao diện người dùng React Vite tới API Express 5 và CSDL SQL Server, bảo vệ an toàn đối với tệp tin tải lên và tuân thủ nghiêm ngặt chuẩn mực phân quyền RBAC.

Bước thực hiện	Lệnh thực thi dòng lệnh (Command)	Mục đích & Đầu ra kiểm tra
1. Khởi động Backend	pnpm run dev	Chạy tại http://localhost:5000

2. Khởi động Frontend	pnpm run dev:frontend	Mở http://localhost:5173
3. Kiểm thử kịch bản Upload	pnpm run test:upload	Kiểm tra kịch bản tải lên avatar
4. Kiểm thử Frontend & CORS	pnpm run test:frontend	Kiểm tra giao diện và kết nối CORS
5. Biên dịch mã nguồn	pnpm build && pnpm build:frontend	Kiểm tra biên dịch TypeScript

Người lập báo cáo: ĐẶNG BẮC NAM *Mã dự án: BACKEND-20261001-DANGBACNAM*